

Basic Probability relationships

(union, intersection, complement, addition law, multiplication law, independent event)

Without Library

```
In [2]: 1 S = [1, 2, 3, 4, 5, 6]
        2 A = [1, 2, 3]
        3 B = [2, 4, 6]
```

```
In [3]: 1 union_ab = A[:]
        2 for element in B:
        3     if element not in union_ab:
        4         union_ab.append(element)
        5 print("Union without set:", union_ab)
```

Union without set: [1, 2, 3, 4, 6]

```
In [19]: 1 def union(a, b):
        2     return a.union(b)
        3
        4 set_a = {1, 2, 3}
        5 set_b = {3, 4, 5}
        6 result = union(set_a, set_b)
        7 result
```

Out[19]: {1, 2, 3, 4, 5}

```
In [8]: 1 intersection_ab = [x for x in A if x in B]
        2 intersection_ab
```

Out[8]: [2]

```
In [21]: 1 def intersection(a, b):
        2     return a.intersection(b)
        3
        4 set_a = {1, 2, 3}
        5 set_b = {3, 4, 5}
        6 result = intersection(set_a, set_b)
        7 result
```

Out[21]: {3}

```
In [10]: 1 complement_a = [x for x in S if x not in A]
          2 complement_a
```

Out[10]: [4, 5, 6]

```
In [23]: 1 def complement(a, universal_set):
          2     return universal_set - a
          3
          4 universal = {1, 2, 3, 4, 5, 6}
          5 set_a = {1, 2, 3}
          6 result = complement(set_a, universal)
          7 result
```

Out[23]: {4, 5, 6}

```
In [11]: 1 P_A = len(A) / len(S)
          2 P_B = len(B) / len(S)
          3 P_A_inter_B = len([x for x in A if x in B]) / len(S)
          4
          5 P_A_union_B = P_A + P_B - P_A_inter_B
          6 P_A_union_B
```

Out[11]: 0.8333333333333334

```
In [26]: 1 def addition_law(p_a, p_b, p_intersection):
          2     return p_a + p_b - p_intersection
          3 p_a = 0.4
          4 p_b = 0.3
          5 p_intersection = 0.1
          6 result = addition_law(p_a, p_b, p_intersection)
          7 result
```

Out[26]: 0.6

```
In [13]: 1 P_A = len(A) / len(S)
          2 P_B = len(B) / len(S)
          3 P_A_inter_B = P_A * P_B
          4 P_A_inter_B
```

Out[13]: 0.25

```
In [28]: 1 def multiplication_law(p_a, p_b_given_a):
          2     return p_a * p_b_given_a
          3
          4 p_a = 0.5
          5 p_b_given_a = 0.3
          6 result = multiplication_law(p_a, p_b_given_a)
          7 result
```

Out[28]: 0.15

```
In [17]: 1 P_A = len(A) / len(S)
          2 P_B = len(B) / len(S)
          3 P_A_and_B = len([x for x in A if x in B]) / len(S)
          4 print(P_A_and_B)
          5 print(P_A * P_B)
```

```
0.16666666666666666
0.25
```

```
In [ ]: 1 if abs(P_A_and_B - P_A * P_B) < 1e-6:
          2     print("A and B are independent.")
          3 else:
          4     print("A and B are dependent.")
```

```
In [30]: 1 def independent_events(p_a, p_b):
          2     return p_a * p_b
          3
          4 p_a = 0.5
          5 p_b = 0.4
          6 result = independent_events(p_a, p_b)
          7 result
```

```
Out[30]: 0.2
```

With Library

```
In [4]: 1 S = [1, 2, 3, 4, 5, 6]
          2 A = [1, 2, 3]
          3 B = [2, 4, 6]
```

```
In [6]: 1 union_ab = set(A).union(set(B))
          2 union_ab
```

```
Out[6]: {1, 2, 3, 4, 6}
```

```
In [20]: 1 union_ab=set_a | set_b
          2 union_ab
```

```
Out[20]: {1, 2, 3, 4, 5}
```

```
In [7]: 1 intersection_ab = set(A).intersection(set(B))
          2 intersection_ab
```

```
Out[7]: {2}
```

```
In [22]: 1 set_a = {1, 2, 3}
          2 set_b = {3, 4, 5}
          3 result = set_a & set_b
          4 result
```

Out[22]: {3}

```
In [9]: 1 complement_a = set(S).difference(set(A))
          2 complement_a
```

Out[9]: {4, 5, 6}

```
In [25]: 1 universal = {1, 2, 3, 4, 5, 6}
          2 set_a = {1, 2, 3}
          3 result = universal - set_a
          4 result
```

Out[25]: {4, 5, 6}

```
In [ ]: 1
```

Discrete Probability Distributions

Binomial Distribution

Without library:

```
In [35]: 1 # Factorial function
2 def factorial(n):
3     result = 1
4     for i in range(2, n+1):
5         result *= i
6     return result
7
8 # Binomial coefficient
9 def nCk(n, k):
10     return factorial(n) // (factorial(k) * factorial(n - k))
11
12 # Binomial Probability Calculation
13 def binomial_pmf(n, k, p):
14     return nCk(n, k) * (p ** k) * ((1 - p) ** (n - k))
15
16 # Parameters
17 n = 10
18 p = 0.5
19 k = 6
20
21 prob_manual = binomial_pmf(n, k, p)
22 print(f"P(X = {k}) (manual) =", prob_manual)
23
```

P(X = 6) (manual) = 0.205078125

```

In [32]: 1 # Factorial
2 def factorial(n):
3     result = 1
4     for i in range(2, n+1):
5         result *= i
6     return result
7
8 # nCk
9 def nCk(n, k):
10     return factorial(n) // (factorial(k) * factorial(n - k))
11
12 # Binomial PMF
13 def binomial_pmf(n, k, p):
14     return nCk(n, k) * (p ** k) * ((1 - p) ** (n - k))
15
16 # Binomial CDF: Sum of PMFs
17 def binomial_cdf(n, k, p):
18     return sum(binomial_pmf(n, i, p) for i in range(0, k + 1))
19
20 # Parameters
21 n = 10
22 p = 0.5
23 k = 6
24
25 cdf_manual = binomial_cdf(n, k, p)
26 print(f"P(X ≤ {k}) (manual) =", cdf_manual)
27

```

$P(X \leq 6)$ (manual) = 0.828125

With library:

```

In [34]: 1 from scipy.stats import binom
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Parameters
6 n = 10      # number of trials
7 p = 0.5     # probability of success
8 k = 6      # number of desired successes
9
10 # Probability of exactly 6 heads
11 prob = binom.pmf(k, n, p)
12 print(f"P(X = {k}) =", prob)
13

```

$P(X = 6)$ = 0.20507812499999999

```
In [31]: 1 from scipy.stats import binom
2
3 n = 10      # number of trials
4 p = 0.5     # probability of success
5 x = 6       # cumulative up to and including this value
6
7 # CDF:  $P(X \leq x)$ 
8 cdf_val = binom.cdf(x, n, p)
9 print(f"P(X ≤ {x}) =", cdf_val)
```

$P(X \leq 6) = 0.828125$

Poisson Distribution

Without Library

```
In [39]: 1 import math
2
3 # Manual PDF
4 def poisson_pmf(k, mu):
5     return (math.exp(-mu) * (mu ** k)) / math.factorial(k)
6
7 # Manual CDF
8 def poisson_cdf(k, mu):
9     return sum(poisson_pmf(i, mu) for i in range(0, k + 1))
10
11 # Parameters
12 mu = 4
13 k = 2
14
15 # Calculate
16 pdf_manual = poisson_pmf(k, mu)
17 cdf_manual = poisson_cdf(k, mu)
18
19 print(f"P(X = {k}) (manual) =", pdf_manual)
20 print(f"P(X ≤ {k}) (manual) =", cdf_manual)
21
```

$P(X = 2) \text{ (manual)} = 0.14652511110987343$

$P(X \leq 2) \text{ (manual)} = 0.2381033055535443$

With Library

```
In [37]: 1 from scipy.stats import poisson
          2
          3 # Parameters
          4 mu = 4
          5 k = 2
          6
          7 # PDF / PMF:  $P(X = 2)$ 
          8 pdf_val = poisson.pmf(k, mu)
          9 print(f"P(X = {k}) =", pdf_val)
         10
         11 # CDF:  $P(X \leq 2)$ 
         12 cdf_val = poisson.cdf(k, mu)
         13 print(f"P(X ≤ {k}) =", cdf_val)
         14
```

$P(X = 2) = 0.14652511110987343$

$P(X \leq 2) = 0.23810330555354436$

Hypergeometric Distribution

Without Library

In [41]:

```

1  import math
2
3  # Combinations function
4  def nCk(n, x):
5      return math.comb(n, x) # or use factorial logic if math.comb not available
6
7  # PDF / PMF
8  def hypergeom_pmf(x, N, r, n):
9      return (nCk(r, x) * nCk(N - r, n - x)) / nCk(N, n)
10
11 # CDF
12 def hypergeom_cdf(x, N, r, n):
13     return sum(hypergeom_pmf(i, N, r, n) for i in range(0, x + 1))
14
15 # Parameters
16 N = 52
17 r = 12
18 n = 5
19 x = 2
20
21 # Manual calculation
22 pdf_manual = hypergeom_pmf(x, N, r, n)
23 cdf_manual = hypergeom_cdf(x, N, r, n)
24
25 print(f"P(X = {x}) (manual) =", pdf_manual)
26 print(f"P(X ≤ {x}) (manual) =", cdf_manual)
27

```

$P(X = 2)$ (manual) = 0.25090036014405764

$P(X \leq 2)$ (manual) = 0.9260504201680673

With Library

```
In [40]: 1 from scipy.stats import hypergeom
2
3 # Parameters
4 N = 52      # population size
5 r = 12      # number of face cards
6 n = 5       # number of draws
7 x = 2       # number of desired face cards in draw
8
9 # PDF / PMF:  $P(X = 2)$ 
10 pdf_val = hypergeom.pmf(x, N, r, n)
11 print(f"P(X = {x}) =", pdf_val)
12
13 # CDF:  $P(X \leq 2)$ 
14 cdf_val = hypergeom.cdf(x, N, r, n)
15 print(f"P(X ≤ {x}) =", cdf_val)
16
```

$P(X = 2) = 0.25090036014405753$

$P(X \leq 2) = 0.9260504201680673$

```
In [ ]: 1
```

Continuous Probability Distributions

Normal Distributions

With Library

```
In [42]: 1 from scipy.stats import norm
2
3 mu = 50
4 sigma = 10
5 x = 60
6
7 # PDF
8 pdf_val = norm.pdf(x, mu, sigma)
9 print(f"PDF at x = {x}:", pdf_val)
10
11 # CDF
12 cdf_val = norm.cdf(x, mu, sigma)
13 print(f"CDF at x = {x}:", cdf_val)
```

PDF at x = 60: 0.024197072451914336

CDF at x = 60: 0.8413447460685429

Without Library

```
In [44]: 1 import math
2
3 # Parameters
4 mu = 50
5 sigma = 10
6 x = 60
7
8 # PDF
9 def normal_pdf(x, mu, sigma):
10     part1 = 1 / (sigma * math.sqrt(2 * math.pi))
11     part2 = math.exp(-((x - mu) ** 2) / (2 * sigma ** 2))
12     return part1 * part2
13
14 # CDF
15 def normal_cdf(x, mu, sigma):
16     z = (x - mu) / (sigma * math.sqrt(2))
17     return 0.5 * (1 + math.erf(z)) # using built-in erf function
18
19 # Manual results
20 pdf_manual = normal_pdf(x, mu, sigma)
21 cdf_manual = normal_cdf(x, mu, sigma)
22 print(pdf_manual)
23 print(cdf_manual)
```

0.02419707245191434

0.8413447460685428

standard Normal Distributions

With Library

```
In [46]: 1 from scipy.stats import norm
2
3 z = 1.5 # standard normal variable
4
5 # PDF
6 pdf_val = norm.pdf(z)
7 print(f"Standard Normal PDF at z = {z}:", pdf_val)
8
9 # CDF
10 cdf_val = norm.cdf(z)
11 print(f"Standard Normal CDF at z = {z}:", cdf_val)
12
```

Standard Normal PDF at z = 1.5: 0.12951759566589174

Standard Normal CDF at z = 1.5: 0.9331927987311419

Without Library

```
In [45]: 1 import math
          2
          3 z = 1.5
          4
          5 # PDF
          6 def standard_normal_pdf(z):
          7     return (1 / math.sqrt(2 * math.pi)) * math.exp(-z ** 2 / 2)
          8
          9 # CDF
         10 def standard_normal_cdf(z):
         11     return 0.5 * (1 + math.erf(z / math.sqrt(2)))
         12
         13 # Manual calculations
         14 pdf_manual = standard_normal_pdf(z)
         15 cdf_manual = standard_normal_cdf(z)
         16 print(pdf_manual)
         17 print(cdf_manual)
```

0.12951759566589174

0.9331927987311419